Project 2

IPT Data Platform

Building a Modern Analytics Infrastructure

Big Data Processing Course

MEI-IoT 2025/2026

Prof. Renato Panda
renato.panda@ipt.pt

Instituto Politécnico de Tomar

Deadline: January 31, 2026

Table of contents

1 Project Overview **3**

2 The Scenario **3**

3 Data Sources **4**
3.1 Provided Sources 4
3.2 Optional Challenge Sources 4

4 Technical Requirements **5**
4.1 Infrastructure (Docker Compose) 5
4.2 Data Pipeline (ELT Pattern) 5
4.3 dbt Models 6
4.4 Dashboard 7

5 Deliverables **7**
5.1 Working Platform 7
5.2 Codebase Structure 7
5.3 Documentation 8
5.4 Short Report (2-3 pages or 3-4 slides) 8

6 Getting Started **8**
6.1 Step 1: Run the Base Setup 8
6.2 Step 2: Add Your Warehouse 9
6.3 Step 3: Build Your Ingestion 9
6.4 Step 4: Transform with dbt 9
6.5 Step 5: Build the Dashboard 9

7 Technical Hints **9**
7.1 Option 1: SQL with dblink (Recommended for Simplicity) 9
7.2 Option 2: COPY with CSV Files 10
7.3 Option 3: Python Scripts 10
7.4 Option 4: Airbyte (Optional, for Exploration) 11
7.5 Integrating File Sources 11

1 Project Overview

In Project 1, you explored data extraction and analysis techniques. Now it's time to think like a **Data Engineer**. Organizations don't run one-time analyses, they build infrastructure that continuously integrates, transforms, and serves data.

IPT has multiple operational systems: in our scenario, apps developed in Software Engineering courses, administrative databases, and spreadsheets. Each system is a **data silo**, isolated and unable to answer questions that span multiple domains.

> **Project Goal:** Build a working data platform that unifies data from multiple IPT systems using the ELT pattern. You will implement data ingestion, transform data with dbt, and create a dashboard that answers real questions for IPT leadership. If done well, this prototype could serve as a foundation for IPT's future analytics infrastructure.

This is a practical project. The goal is not to build a production-ready system, but to understand how the pieces fit together: sources, ingestion, warehouse, transformations, and visualization.

2 The Scenario

You are part of a small data team tasked with building IPT's first analytics platform. The leadership wants to answer questions like:

- "Which rooms have the most maintenance issues AND the lowest utilization?"
- "Are students attending classes in rooms with known problems?"
- "How quickly are maintenance issues being resolved by building?"
- "What schools, courses, course units have the highest and lowest attendance rates?"
- "Are there correlations between room conditions (issues) and student attendance?"
- "How many students are enrolled in courses? How many actually attend classes?"
- "Regarding quality audits, which processes have the most non-conformities?"
- "How many publications and citations does IPT have this year? Who are the most prolific researchers?"
- "How many maintenance issues were reported vs resolved last month?"
- "What is the average time to resolve issues by type or building?"
- "What is the attendance drop-off from first to last class in each course unit?"

These are just examples of questions IPT leadership, schools, and departments want to answer. Currently, nobody can answer these questions without manually exporting and joining data.

Your mission: Build a prototype data platform that demonstrates how similar questions can be answered automatically.

3 Data Sources

A base setup is provided with four PostgreSQL databases simulating IPT operational applications. Each database comes with a schema and Python generators that create realistic[1] sample data covering the first semester of 2025/2026.

3.1 Provided Sources

1. **Issues Reporting App** – Maintenance issues reported by staff and students across IPT buildings
2. **Room Reservations App** – Room bookings for classes, exams, meetings, and events
3. **Summaries & Attendance App** – Class summaries and student attendance records
4. **SIGQ Platform** – Quality management system with documents, audits, and corrective actions

3.2 Optional Challenge Sources

The previous sources cover the core operational data. If you want to go further, you can integrate additional sources:

- **Your P1 Project Data** – If you built a data extraction pipeline in Project 1, you can connect it here. For example, if you scraped student applications, that could become another source in your warehouse (the csvs).
- **OpenAlex Research Metrics** – IPT's research output is available through the OpenAlex API.
  - You can fetch publication counts, citation metrics, and researcher information institution wise directly from https://api.openalex.org/institutions/i4210126619. This is a good example of API-based ingestion and addresses a real gap in IPT's analytics.

[1] The data is generated using Python scripts that produce plausible but fictional records, or at least I tried to. Feel free to modify them to suit your needs.

  - In addition, if you want to fetch a person at a time, the IPT has two PDF files where you can find the list of researchers and their ORCID IDs: https://portal2.ipt.pt/pt/ipt/estrutura_organica/instituto_politecnico_de_tomar/unidades_funcionais/unidades_de_apoio/servicos_centrais/divisao_de_recursos_humanos/, and can then query OpenAlex for each ORCID https://api.openalex.org/authors/https://orcid.org/0000-0003-2539-5590

- Other sources can be provided as files if you ask in advance, for example:
  - **Lab Inventory** (LAB-LE) – Electrical equipment and materials in labs (Excel files)
  - **Pedagogical Questionnaires** – Student feedback surveys (SPSS or CSV files)

These are optional, do not try to integrate them all. Start with the base and later consider adding optional sources if time allows. Integrating a non-database source (files, APIs) demonstrates a more complete understanding of data ingestion.

4 Technical Requirements

4.1 Infrastructure (Docker Compose)

The base setup provides:

- **Source databases**: Four PostgreSQL instances with sample data (issues, reservations, attendance, SIGQ)
- **Adminer**: Web interface for browsing databases

You need to extend the compose file to add:

- **Data Warehouse**: A centralized PostgreSQL database where all data is combined
- **Metabase** (or similar): Dashboard and BI tool

Your final platform should start with `docker compose up -d`, followed by your ingestion script and dbt transformations properly documented in a `README.md`.

4.2 Data Pipeline (ELT Pattern)

Implement the **ELT** pattern as covered in class:

1. **Extract & Load**: Pull data from source databases into the warehouse's `raw` schema

2. **Transform**: Use dbt to create staging and mart models (cleaned, standardized, and business-ready tables, joining multiple sources)
3. **Serve**: Connect Metabase to the transformed tables

The ingestion step (Extract & Load) can be implemented in several ways:

- **SQL scripts with dblink**: Query source databases directly from the warehouse using PostgreSQL's `dblink` extension. Simple and requires no additional tools.
- **Bash scripts with psql**: Export data with `COPY TO` and import with `COPY FROM`. Works well for batch loads.
- **Python scripts**: Use `psycopg2` or `sqlalchemy` to read from sources and write to the warehouse. More flexible for complex logic.
- **Airbyte**: A dedicated data integration tool with a web UI. More setup, but handles schema changes and incremental syncs automatically.

> **Keep it Simple:** For this project, a bash script or SQL with dblink is enough. If you want to explore Airbyte, it's a good learning experience, but don't let the tooling distract from the core goal: understanding how data flows from sources to warehouse to dashboards. In production, teams often start simple and evolve their pipelines over time.

4.3 dbt Models

Your dbt project should include:

- **Sources**: Definitions for raw tables (`sources.yml`)
- **Staging models**: Clean[2] and standardize each source (`stg_*.sql`)
- **Mart models**: Business-ready tables[3] that JOIN multiple sources (`mart_*.sql`)
- **At least one mart** that combines data from various sources[4]. Still, don't overcomplicate it. Focus on a few key models that demonstrate your understanding of dbt and the ELT process.

[2] Clean means handling missing values, correcting data types, and fixing inconsistencies. For example, converting date strings to proper date types, filling missing values with defaults, and standardizing categorical values.

[3] Business-ready means tables that are optimized for analysis, often combining and aggregating data from multiple sources, and structured to answer specific business questions. For example, a mart table might join data from attendance, issues and reservations to analyze room utilization.

[4] For example, a mart that combines summaries, attendance, and issues to analyze the impact of room conditions on student attendance.

4.4 Dashboard

Build a Metabase dashboard with **at least 5 visualizations** that answer real[5] questions:

- Mix of tables, charts, and numbers
- If possible, check if one visualization using joined data from multiple sources can be created
- Clear titles and descriptions

5 Deliverables

5.1 Working Platform

Your submission should be a complete, self-contained folder. Starting from the provided base setup, a typical workflow would be:

```
docker compose up -d
./scripts/generate_and_load.sh   # Load sample data into source DBs
./scripts/ingest.sh              # Your script to load into warehouse
cd dbt && dbt run                # Transform the data
```

After these commands, Metabase should have data ready to visualize. Consider adding a `run.sh` script that automates all steps for ease of use. Extra: how could you schedule regular refreshes (e.g., daily)?

5.2 Codebase Structure

```
project/
├── docker-compose.yml             # Extended with warehouse + metabase
├── source_databases/              # Provided
│   ├── issues_db/init.sql
│   ├── reservations_db/init.sql
│   ├── attendance_db/init.sql
│   └── sigq_db/init.sql
├── generators/                    # Provided
│   └── *.py
├── scripts/
```

[5] Real means based on actual data and business questions relevant to IPT leadership, not just arbitrary charts. Obviously the data is fictional.

```
│   ├── generate_and_load.sh        # Provided
│   └── ingest.sh                   # Your ingestion script
├── dbt/                            # Your dbt project
│   ├── dbt_project.yml
│   ├── profiles.yml
│   └── models/
│       ├── sources.yml
│       ├── staging/
│       └── marts/
└── README.md
```

5.3 Documentation

A `README.md` that includes:

- Brief description of the platform
- How to run it (step-by-step)
- Description of each data source (schema, purpose)
- List of dbt models and what they calculate
- Screenshots of the dashboard
- Any design decisions or assumptions made

5.4 Short Report (2-3 pages or 3-4 slides)

A brief document or presentation explaining:

- The architecture decisions you made and why
- Challenges faced during implementation
- What insights can IPT leadership gain from your dashboard
- How you would extend this platform in a real scenario

6 Getting Started

6.1 Step 1: Run the Base Setup

Start with the provided `p2-ipt-data-platform` folder:

```
cd p2-ipt-data-platform
docker compose up -d
./scripts/generate_and_load.sh
```

This gives you four databases with realistic sample data. Use Adminer at http://localhost:8080 to explore the schemas and understand what data is available (check compose for db credentials and ports).

6.2 Step 2: Add Your Warehouse

Extend the `docker-compose.yml` to add a warehouse database. This is where all data will be combined. You might also want to add Metabase at this point.

6.3 Step 3: Build Your Ingestion

Create a script that extracts data from the four source databases and loads it into your warehouse. You can use `psql` with the `dblink` extension, `COPY` commands, Python scripts, or a tool like Airbyte. The key is to land the raw data in a `raw` schema without transforming it.

6.4 Step 4: Transform with dbt

Set up a dbt project that reads from your `raw` schema and creates:

- Staging models that clean and standardize each source
- Mart models that join data across sources to answer business questions
- Tip: several approaches are possible but start by understanding each source, then build staging models, and finally create marts that combine data.

6.5 Step 5: Build the Dashboard

Connect Metabase to your warehouse and create visualizations that answer real questions. Focus on clarity and usefulness.

7 Technical Hints

7.1 Option 1: SQL with dblink (Recommended for Simplicity)

PostgreSQL's `dblink` extension lets the warehouse query source databases directly. This is the simplest approach when all databases are in the same network:

```
-- Enable the extension in your warehouse
CREATE EXTENSION IF NOT EXISTS dblink;

-- Create a raw table by pulling from a source database
CREATE TABLE raw.issues AS
SELECT * FROM dblink(
    'host=issues_db dbname=issues_app user=issues_user password=issues_pass',
    'SELECT id, title, status, reported_at FROM issues'
) AS t(id int, title varchar, status varchar, reported_at timestamp);
```

The main advantage is that extract and load happen in a single SQL statement. You can wrap these in a shell script and run it whenever you want to refresh the data.

7.2 Option 2: COPY with CSV Files

Another approach is to export data to CSV files and import them:

```
# Export from source
PGPASSWORD=issues_pass psql -h localhost -p 5432 -U issues_user -d issues_app \
    -c "COPY issues TO STDOUT WITH CSV HEADER" > issues.csv

# Import to warehouse
PGPASSWORD=dbt_pass psql -h localhost -p 5434 -U dbt_user -d warehouse \
    -c "COPY raw.issues FROM STDIN WITH CSV HEADER" < issues.csv
```

This is useful if you need to inspect or transform the data between export and import. The example provided during class uses this method.

7.3 Option 3: Python Scripts

For more complex logic, you can write Python scripts using `psycopg2`:

```
import psycopg2

# Connect to source and warehouse
source = psycopg2.connect(host='localhost', port=5432, ...)
warehouse = psycopg2.connect(host='localhost', port=5434, ...)

# Read from source, write to warehouse
with source.cursor() as cur:
```

```
    cur.execute('SELECT * FROM issues')
    rows = cur.fetchall()

# Insert into warehouse...
```

This gives you full control but requires more code.

7.4 Option 4: Airbyte (Optional, for Exploration)

Airbyte is an open-source data integration platform. It provides a web UI to configure connections between sources and destinations, handles schema detection, and supports incremental syncs.

To try it, check some of the Airbyte documentation - Quickstart or online tutorials like this one.

Again, Airbyte is super overkill for this project, but if you want to explore modern data integration tooling, it's worth experimenting with at the end and after you have a working pipeline. It's particularly useful for non-database sources like APIs or files.

7.5 Integrating File Sources

If you want to add Excel, CSV, or other file sources:

- **Simple approach**: Convert to CSV and load with `COPY` or a Python script
- **With Airbyte**: Use the appropriate connector (e.g., CSV, Excel) to read files and load into the warehouse.

Final Advice: Start simple. Get a working pipeline with 2 sources first, then expand. A simple, working platform is worth more than an ambitious but broken one. Remember: the goal is understanding the architecture, not impressing with complexity. Also check the example solution provided in class for reference.